

Programming Domains :

Scientific:

Large number of floating point computations

- Fortran (still used)
- ALGOL 60

Business:

Produce reports, use decimal numbers and characters

- COBOL

Artificial intelligence:

Symbols , consisting of names rather than numbers, are manipulated

- LISP
- Scheme
- Prolog

System programming :

systems SW(OS, support tools), must be efficient because of continuous use

- C, UNIX OS written in C

Web Software:

- markup, XHTML
- scripting, PHP
- general purpose, JavaScript

• Language Evaluation Criteria:

1- Readability: the ease with which programs can be read and understood, it most Important measure of the quality of programs mainly because if maintenance and cost

Simplicity:

- a manageable set of features and constructs :
large number of basic constructs more difficult to learn

- feature multiplicity:
having more than one way to do an operation, ex: increment an integer variable in java
- operator overloading:
an operator symbol that has more than one meaning, ex: '+' used for both integer and floating point addition

Note: too much simplicity makes program less readable, ex: assembly

Orthogonality:

language constructs should not behave differently in different contexts, and the context of use should not cause unexpected restrictions or behavior

Most orthogonal language: Algol 68

- trade-off between orthogonality and simplicity is a central decision in language design
- Note: too much orthogonality can cause a problems

Control statement:

goto statement severely reduces readability
Ex: early version of Fortran

Data types and structures:

the presence of meaningful data types aids readability

Ex: variable = 1 (unclear, no Boolean type in the language)

variable = true (clear, Boolean data type in the language)
Ex2: record data type to provide a method to represent variable records

Syntax design:

- Identifier length : affect readability
flexible composition, it would be better if there is no restriction on the length
- Special words & methods of compound statements:
ex: difficult to determine which group is ended when an "end " or "}" appears
Note: Ada language uses "end if" and "end loop" to terminate compound construct (to make which group is ended more clear)
- Form and meaning(semantic):
ex: meaning of static in C depends on the context of its appearance
ex2: UNIX syntax does not suggest semantic (must be descriptive constructs & meaningful keywords)

2- Writability: the ease with which a language can be create programs

All characteristics that affected in readability

Support for abstraction:

The ability to define and use complex structures or operation on ways that allow details to be ignored, ex: sub program

Expressivity:

Set of relatively convenient ways to specifying operations

Ex: `count++` better than `count = count + 1`

Ex2: inclusion of `for` statement

Note: is not reasonable to compare writability of two languages when one was designed for that application, ex: VB for writing GUI program while C for writing system program, such as OS

3- Reliability: performance of a program to its specifications under all conditions

All characteristics that affected in readability and writability:

- The easier a program to write, the more likely it is to be correct
- Readability affects realizability in the writing and maintenance phases of the life cycle
- Programs difficult to read are difficult to write and modify

Type checking:

Testing for type errors, because run-time type checking is expensive, compile-time type checking (as in Java) is more desirable

Exception handling:

Intercept run-time errors and take corrective measures
Ex: Java, C++, Ada

Aliasing:

Having two or more distinct names to access the same memory cell
Ex: two pointers point to the same variable

4- Cost:

Training
programming
to use
language

Writing programs

Compiling programs

Executing programs:

Trade-off between
compilation cost &
execution speed
Note:
Optimization: collection of
techniques that compiler
may use to decrease the
size or increase the
execution speed
> Increase the execution
speed : use the
optimization
> Increase the compilation
speed: doesn't use the
optimization

language
implementation
system:
Availability of free
compilers
Ex: Java

Reliability:
Poor reliability leads
to high costs

Maintaining
programs

Note: most important contributors to language cost are: development, maintenance, and reliability

5- Others:

Portability:

The ease with which
programs can be
moved from one
implementation to
another

Generality:

The applicability to wide
range of applications

Well-define:

The completeness and
precision of the language's
official definition

Computer Architecture Influence

Computer architecture:

Most languages are developed around well-known computer architecture (Von Neumann architecture)

- Imperative languages: dominant because of Von Neumann computers

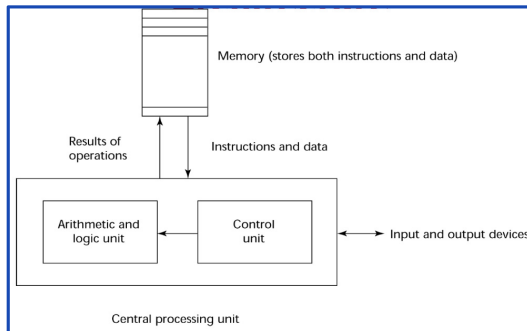
> Concept of Von Neumann :

1- Data and programs stored in memory

2- CPU is separate from memory

3- Instructions and data are transmitted (piped) from memory to CPU & results of operations in CPU moved back to memory

4- Basis (features) : variables model memory cells, assignment statement model piping, repetition), and iteration is efficient



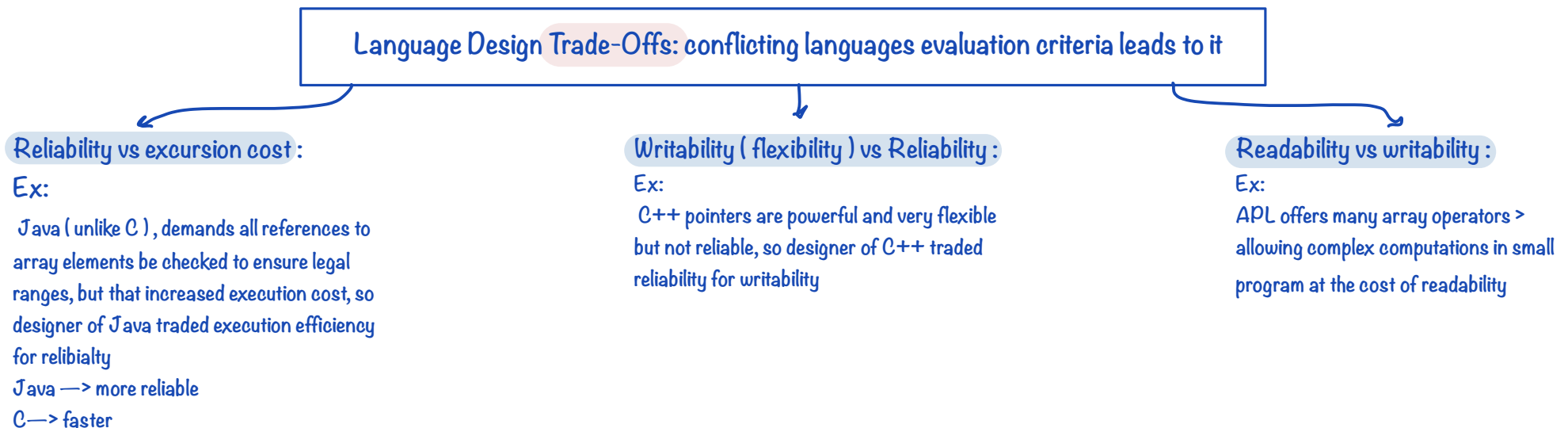
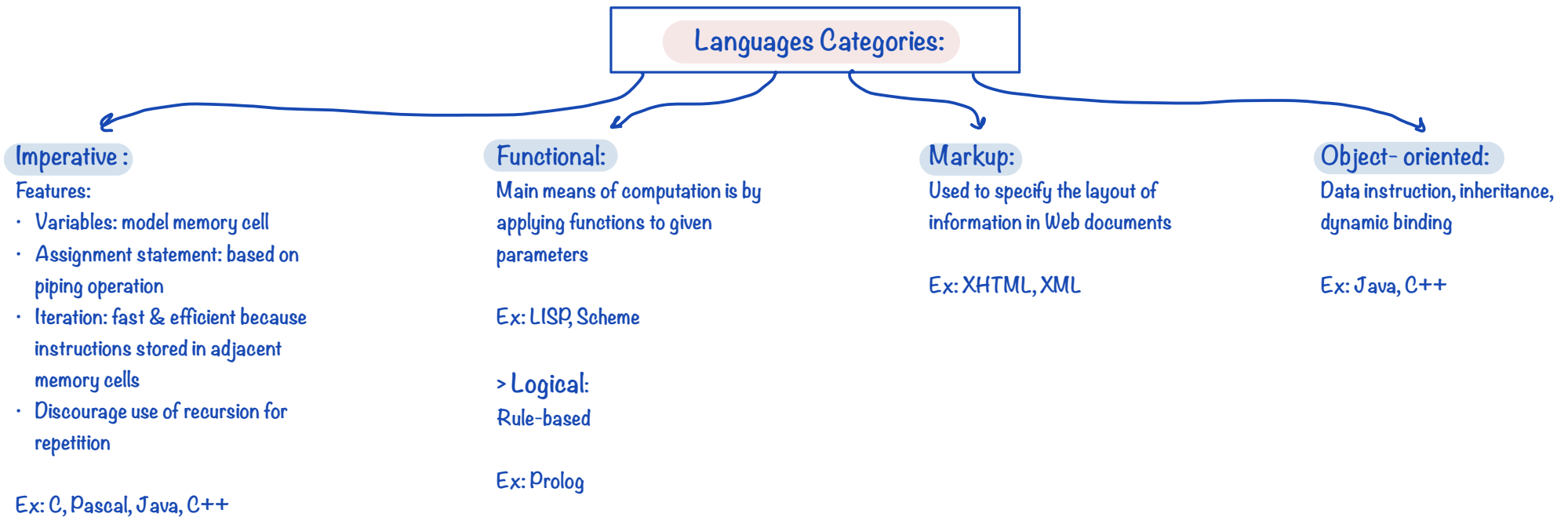
- Fetch-execute-cycle on Van Neumann :

```
initialize the program counter (address of next instruction)
repeat forever
    fetch the instruction pointed by the counter
    increment the counter
    decode the instruction
    execute the instruction
end repeat
```

Programming design methodologies :

- Procedure-oriented programming: the opposite of data-oriented programming, and its methods have not been abended
- Data-oriented programming: focus on data design & abstract data types, and its methods dominate SW development

فيه معلومات زياده في سلايد ٢٦ ، كاتبة عند التفصيل تبعها انه بس قراءة ما بييجي ، بس مدرسي
عني سحبت عليه :



Note: Choosing constructs and features when designing a programming language requires many compromises and trade-offs

Programming Environment :

UNIX:

An older operating system programming environment and tool collection,
Used through a GUI that runs on top of UNIX

Borland JBuilder:

An integrated development environment for Java

Microsoft Visual Studio.NET:

A large visual environment, used to program in C#, Visual BASIC.NET, Jscript, J#, C++

Languages and environment Summary

ALGOL:

- Scientific (60)
- Most orthogonal (68)

Fortran:

- Scientific
- Early version of it use goto statement

COBOL:

- Business
- Use compilers

Prolog:

- Artificial intelligence
- Logic

Ada:

- Use end if and end loop
- Use compilers

Java:

- Imperative
- Object-oriented
- Widely used because of its free compiler
- Designer traded execution efficiency for reliability
- Example for hybrid implementation system
- Borland JBuilder

Scheme:

- Artificial intelligence
- Functional

LISP:

- Artificial intelligence
- Functional

Pascal:

- Imperative

APL:

- Designer traded readability for writability
- Example for pure interpretation

C:

- Written UNIX OS
- Use compilers
- Syntax suggest semantic
- Imperative
- For writing program, such as OS
- Support aliasing
- Designer traded reliability for execution cost

C++:

- Imperative
- Object-oriented
- Designer traded of reliability for writability
- Microsoft Visual Studio.NET

JavaScript:

- Web software (general-purpose)

Perl:

- Example for Hybrid implementation system

XHTML:

- Web software (Markup)

C#:

Microsoft Visual Studio.NET

J#:

Microsoft Visual Studio.NET

Visual BASIC.NET:

Microsoft Visual Studio.NET

VB:

- For writing GUI program

PHP:

- Web software (scripting)

UNIX:

- Syntax does not suggest semantic
- Older operating system programming environment and tool collection, used through a GUI

XML:

- Web software (Markup)

Assembly:

- Model of simplicity